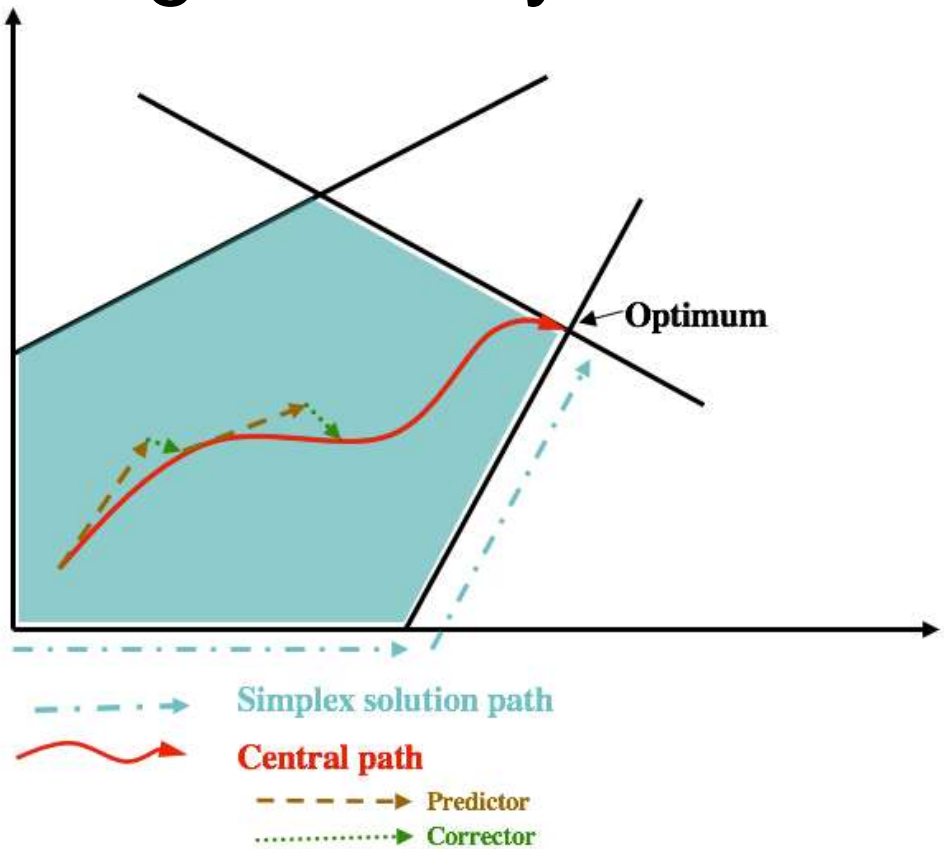
Formulación

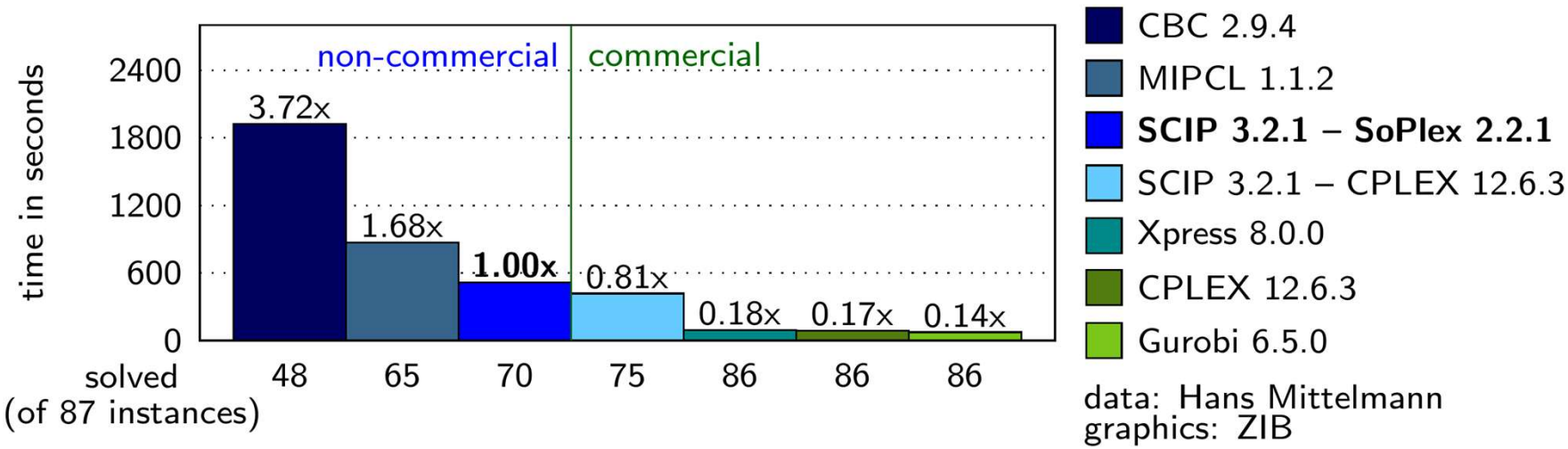
¿ Que hay detras de los Solvers ?



LP y MIP

. . .

Solvers


time in seconds
2400
1800
1200
600
0
non-commercial
commercial
3.72x
1.68x
1.00x
0.81x
0.18x
0.17x
0.14x
solved
(of 87 instances)
48
65
70
75
86
86
86
CBC 2.9.4
MIPCL 1.1.2
SCIP 3.2.1 – SoPlex 2.2.1
SCIP 3.2.1 – CPLEX 12.6.3
Xpress 8.0.0
CPLEX 12.6.3
Gurobi 6.5.0
data: Hans Mittelmann
graphics: ZIB

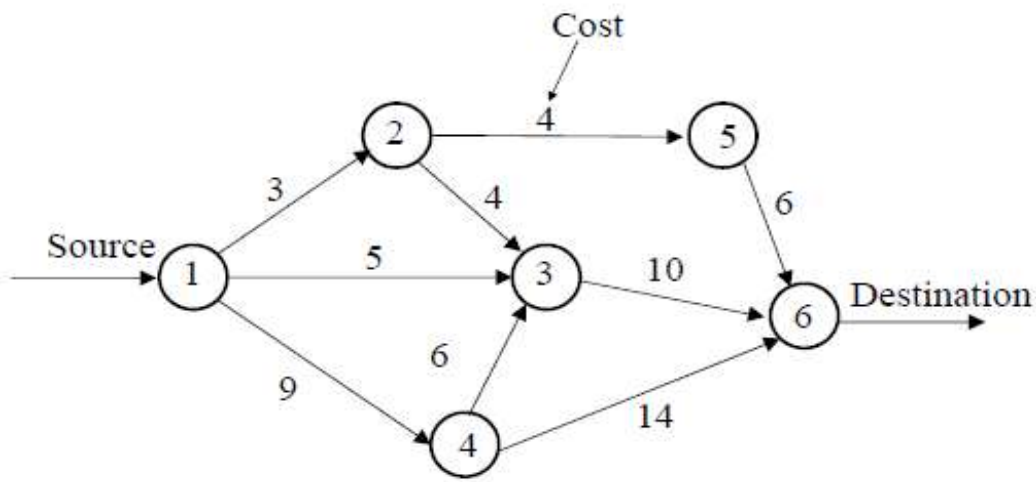
• • •

Gurobi	Cplex
Fundado por los creadores originales de CPLEX. Empresa privada y ágil, 100% enfocada en la optimización matemática. **Frecuentemente el líder en benchmarks independientes.** Conocido por su velocidad y por empujar constantemente los límites del rendimiento en cada nueva versión. **Su API de Python (gurobipy) es ampliamente elogiada** por ser moderna, intuitiva y muy "Pythonic". Facilita la creación de modelos complejos.	El solver pionero de la industria. Propiedad del gigante tecnológico **IBM**, integrado en un ecosistema de productos más amplio. **Extremadamente rápido y robusto.** Un competidor de primer nivel que a veces lidera en ciertos tipos de problemas. El rendimiento es muy cercano al de Gurobi. Ofrece APIs muy potentes y maduras (Python, C++, Java, etc.). La integración puede ser más profunda si ya se utiliza el ecosistema de software de IBM.

Solver	Open Source	Ease of Use	Speed	Scalablity	Best for
PuLP	Yes	4/5.	2/5.	2/5.	Beginners, small problems, prototyping
CPLEX	No (Free for academia)	3/5.	5/5.	5/5.	Industrial, large-scale, logistics
Gurobi	No (Free for academia)	4/5.	5/5.	5/5.	Finance, scheduling, advanced MIP
OR-Tools	Yes	2/5.	4/5.	3/5.	Routing, logistics, open-source projects
Pymono	Yes	3/5.	3/5.	4/5.	Research, complex constraints, engineering

	LP	MILP	QP	MIQP	convex (MI)QCP	non-convex (MI)QCP	Conic	MINLP	MP
Gurobi	✅	✅	✅	✅	✅	✅	✅	✅	✅
FICO XPRESS	✅	✅	✅	✅	✅	✅	✅	✅	✅
IBM CPLEX	✅	✅	✅	✅	✅	❌	✅	❌	✅
COPT	✅ [1]	✅	✅	✅	✅	❌	✅	❌	✅
MOSEK	✅	✅	✅	✅	✅	❌	✅ [2]	❌	✅
NVIDIA cuOpt	✅ [1]	✅ [1]	❌	❌	❌	❌	❌	❌	✅
HiGHS	✅ [1]	✅	✅	❌	❌	❌	❌	❌	✅
CBC	✅	✅	✅	✅	❌	❌	❌	❌	✅
SCIP	✅	✅	✅	✅	✅	✅	✅	✅	✅
GCG	✅	✅	✅	✅	✅	✅	✅	✅	✅

Retomemos...



• • •

Conjuntos y parámetros de la red

El grafo está compuesto por 6 nodos, a partir de ellos se configura una red particular establecida por las relaciones de arcos entre ellos.

$$N = \{1,2,3,4,5,6\}$$

- Sea C_{ij} el costo asociado a recorrer el arco desde i a j.

Cij	**1**	**2**	**3**	**4**	**5**	**6**
1	0	3	5	9	0	0
2	0	0	4	0	4	0
3	0	0	0	0	0	10
4	0	0	6	0	0	14
5	0	0	0	0	0	6
6	0	0	0	0	0	0

- Sea CAP_{ij} la capacidad de flujo del arco origen i al origen destino j.

CAPij	1	2	3	4	5	6
1	0	25	100	70	0	0
2	0	0	30	0	15	0
3	0	0	0	0	0	200
4	0	0	60	0	0	30
5	0	0	0	0	0	150
6	0	0	0	0	0	0

- Sea V el flujo que debe enviarse desde el origen al destino. Luego V=180

Las Variables de decision...

Sea x_{ij} la cantidad de flujo de tráfico desde el nodo i hasta el nodo j. De esta forma las variables que se tienen son:

- $x_{12}: Cantidad\ de\ flujo\ de\ tráfico\ desde\ el\ nodo\ 1\ hasta\ el\ nodo\ 2$
- $x_{25}: Cantidad\ de\ flujo\ de\ tráfico\ desde\ el\ nodo\ 2\ hasta\ el\ nodo\ 5$
- $x_{56}: Cantidad\ de\ flujo\ de\ tráfico\ desde\ el\ nodo\ 5\ hasta\ el\ nodo\ 6$
- $x_{13}: Cantidad\ de\ flujo\ de\ tráfico\ desde\ el\ nodo\ 1\ hasta\ el\ nodo\ 3$
- $x_{36}: Cantidad\ de\ flujo\ de\ tráfico\ desde\ el\ nodo\ 3\ hasta\ el\ nodo\ 6$
- $x_{14}: Cantidad\ de\ flujo\ de\ tráfico\ desde\ el\ nodo\ 1\ hasta\ el\ nodo\ 4$
- $x_{46}: Cantidad\ de\ flujo\ de\ tráfico\ desde\ el\ nodo\ 4\ hasta\ el\ nodo\ 6$
- $x_{43}: Cantidad\ de\ flujo\ de\ tráfico\ desde\ el\ nodo\ 4\ hasta\ el\ nodo\ 3$
- $x_{23}: Cantidad\ de\ flujo\ de\ tráfico\ desde\ el\ nodo\ 2\ hasta\ el\ nodo\ 3$

Nota: Son 36 variables x_{ij}, las demás variables posibles de x_{ij} son igualadas a cero.

Función objetivo

$$Min\ z = \sum_{i \in N} \sum_{j \in N} C_{ij} * x_{ij}$$

Flujo que se debe garantizar desde el origen

$$\sum_{j \epsilon J} x_{1j} = V$$

Nota: Por balance, todo lo que entra debe salir del sistema.

$$\sum_{i \epsilon I} x_{i6} = V$$

Restricciones de balance

$$\forall i \epsilon N: i > 1, i < 6$$

Observe que solo los nodos intermedios tienen entradas y salidas

$$\sum_{j \in N} x_{ij} = \sum_{j \in N} x_{ji}$$

$$\sum_{j \in N} x_{ij}$$

Esta ecuación expresa la suma de todas las salidas del nodo i
Todos los arcos que tienen origen en i.

$$\sum_{j \in N} x_{ji}$$

Esta ecuación expresa la suma de todas las entradas al nodo i
Todos los arcos que tienen destino en i.

Restricciones de capacidad y No negatividad

$$\forall i \epsilon N, \forall j \epsilon N \qquad x_{ij} \leq CAP_{ij}$$

$$\forall i \epsilon N, \forall i \epsilon N \qquad x_{ij} \geq 0$$

Creación de tablas de importación de datos

- Con el fin de sistematizar el modelo de programación lineal se crean tablas de datos externas que se incorporaran como datos de entrada.
- Esto permite modificar los datos sin la necesidad de cambiar el modelo.
- Las tablas deben tener el formato de separación de datos por comas o csv.

```
Origen,Destino,Costo,Capacidad
1,1,0,0
1,2,3,25
1,3,5,100
1,4,9,70
1,5,0,0
1,6,0,0
2,1,0,0
2,2,0,0
2,3,4,30
2,4,0,0
2,5,4,15
2,6,0,0
3,1,0,0
3,2,0,0
3,3,0,0
3,4,0,0
3,5,0,0
3,6,10,200
4,1,0,0
4,2,0,0
4,3,6,60
4,4,0,0
4,5,0,0
4,6,14,30
5,1,0,0
5,2,0,0
5,3,0,0
5,4,0,0
5,5,0,0
5,6,6,150
6,1,0,0
6,2,0,0
6,3,0,0
6,4,0,0
6,5,0,0
6,6,0,0
```

Grafos → NetworkX

Se usa ampliamente para la creación, manipulación y estudio de la estructura, dinámica y funciones de grafos complejos

Con networkx se puede trabajar:

Grafos dirigidos y no dirigidos.
Grafos con pesos, capacidades, costos, etc.
Algoritmos clásicos como búsqueda de caminos, detección de ciclos, flujo máximo, etc.
Visualización básica de grafos

https://networkx.org/en/

Programación en Python: 9 pasos

Paso 1: Instalar PuLP
pip install pulp

Paso 2: Importar librerías
```
from pulp import *
from pandas import DataFrame
```

Paso 3: Definición de Conjuntos
```
Nodos = list(range(0,6))
print(Nodos)
```

Paso 4. Parámetros
```
Capacidad=[[0,25,100,70,0,0],[0,0,30,0,15,0],[0,0,0,0,0,200],[0,0,60,0,0,30],[0,0,0,0,0,150],[0,0,0,0,0,0]]
Costo=[[0,3,5,9,0,0],[0,0,4,0,4,0],[0,0,0,0,0,10],[0,0,6,0,0,14],[0,0,0,0,0,6],[0,0,0,0,0,0]]
```

Paso 5. Creación del problema
```
prob_fm=LpProblem('MinCost',LpMinimize)
```

Paso 6: Creación de variables
```
arcos=[(i,j) for i in Nodos for j in Nodos]
print(arcos)
var_cant=LpVariable.dicts("CantAEnviar",(Nodos,Nodos),lowBound=0,cat="Continuous")
```

Programación en Python

Paso 7: Función Objetivo y restricciones

```
prob_fm += lpSum(var_cant[i][j]*Costo[i][j] for i in Nodos for j in Nodos)
#Restricciones
#Flujo que se debe garantizar
for j in Nodos:
  prob_fm += lpSum(var_cant[0][j] for j in Nodos)==180
#Capacidad
for i in Nodos:
  for j in Nodos:
    prob_fm += var_cant[i][j]<=Capacidad[i][j]
#Balance
for k in Nodos:
  if k>0 and k<5:
    prob_fm+= lpSum(var_cant[i][k] for i in Nodos)==lpSum(var_cant[k][j] for j in Nodos)
```

Paso 8. Se resuelve el problema

```
status=prob_fm.solve()
print("Status: ", LpStatus[prob_fm.status])
```

Paso 9: Imprimir la solución

```
for var in prob_fm.variables():
  if var.value() > 0:
    print(var,'=', value(var))
print("El flujo Mín-Costo enviado es: " + format(value(prob_fm.objective)))
```

Solución

```
CantAEnviar_0_1 = 25.0
CantAEnviar_0_2 = 100.0
CantAEnviar_0_3 = 55.0
CantAEnviar_1_2 = 10.0
CantAEnviar_1_4 = 15.0
CantAEnviar_2_5 = 135.0
CantAEnviar_3_2 = 25.0
CantAEnviar_3_5 = 30.0
CantAEnviar_4_5 = 15.0
El flujo Mín-Costo enviado es: 3180.0
```